AI-Driven Self-Healing Test Automation: A DOM-Similarity and Machine-Learning Framework for Resilient Web UI Regression Testing

Vijay Javvadi
Independent Researcher
AI-Driven Test Automation and Intelligent Quality Engineering
jvijayprasad@gmail.com

***Abstract*—Modern software systems evolve rapidly, and automated user interface (UI) test scripts frequently fail due to minor front-end changes, dynamic element identifiers, or evolving application structures. Traditional automation frameworks require continuous manual maintenance to update broken locators and test scripts, resulting in increased maintenance cost, slower release cycles, and reduced confidence in regression coverage. This paper proposes an AI-driven self-healing test automation framework capable of automatically detecting, diagnosing, and repairing failing test cases caused by UI changes. The framework integrates a structured locator metadata store, Document Object Model (DOM) similarity analysis, heuristic candidate ranking, and a lightweight machine-learning fallback to dynamically identify alternative elements when original locators become invalid. At runtime, the framework monitors historical execution data and page-structure patterns to generate predictive locator models that enable automated recovery during test execution. When a test step fails because of an element change, the framework evaluates multiple candidate elements using structural similarity metrics, attribute matching, and machine-learning-assisted ranking to determine the most probable replacement element. To evaluate the approach, we constructed a reproducible, mutation-based experimental pipeline that derives five progressively mutated versions of a representative web application, applying seven mutation categories—identifier change, attribute removal, DOM wrapping, element reordering, class change, text change, and placeholder change—across fifty-two total mutation events. Nine Selenium-based test suites were executed across all versions, producing a dataset of 1,600 experiment rows, 950 baseline-successful steps, 2,400 broken locator events, 1,550 healed events, and 850 unresolved failures. The empirical evaluation yields a mean healing success rate (HSR) of 64.58% with a Wilson 95% confidence interval of [62.65%, 66.47%], a locator recovery accuracy of 64.58%, and a false healing rate of 35.42%. A denominator decomposition further shows that 100% of broken events produced a ranked candidate, meaning the 35.42% unresolved share reflects a *recovery-executor* bottleneck (the ranked candidate could not be driven in the live DOM) rather than a *ranking* bottleneck. Per-class recovery is highly asymmetric: `id_change` mutations recover at 86.2%, while `attribute_removed` mutations recover at only 31.6%, a 55-point spread that directly explains the aggregate number. Beyond aggregate metrics, the analysis reports per-mutation, per-version, per-test, and cumulative-recovery behavior, revealing which UI change patterns remain most challenging for automated recovery. The results show that AI-driven locator recovery substantially reduces manual automation maintenance while significantly improving test execution continuity across evolving application versions. The research highlights the potential of intelligent automation frameworks to transform traditional test automation into adaptive testing systems capable of maintaining long-term reliability in rapidly evolving software applications.**

***Index Terms*—Self-healing automation, UI testing, Selenium, DOM similarity, machine learning, web test resilience, locator recovery, mutation testing, regression testing, intelligent quality engineering**

I. Introduction

Automated web UI testing is a foundational quality-assurance activity in modern software engineering. Functional regression suites are routinely executed in continuous integration and continuous delivery (CI/CD) build pipelines to validate navigation flows, input handling, shopping-cart behavior, authentication scenarios, and checkout interactions. In principle, UI automation reduces manual test effort, improves repeatability, and increases release confidence. In practice, however, automated UI tests remain strikingly sensitive to changes in front-end implementation details. Industry surveys and developer studies consistently identify locator maintenance as one of the most expensive and time-consuming activities in large regression environments, and a non-trivial share of "broken" test runs in production CI pipelines do not reflect any defect in application behavior. Instead, they indicate that an element identifier, attribute, or DOM path has shifted.

The central source of this sensitivity is *locator fragility*. Selenium-based tests identify interface elements through identifiers such as XPath expressions, CSS selectors, element IDs, placeholder values, and text attributes. These locators encode strong assumptions about the structure of the current DOM. When a front-end developer changes element attributes, inserts wrapper containers, replaces CSS classes, or reorganizes page hierarchy, tests may fail even when the business behavior remains correct. Thus, a significant fraction of UI test failures represent automation maintenance issues rather than true application defects. This distinction matters because failures of the former kind cost engineering time without improving product quality: an engineer must reproduce the failure, inspect the evolved DOM, recover the intended element, and reissue a locator that re-encodes the same assumption into a new context.

This problem becomes more severe in rapidly changing web systems. Teams practicing continuous delivery frequently

modify front-end frameworks, component libraries, and responsive layouts. A locator that was valid in one release may fail in the next release not because the user journey is broken, but because the DOM no longer matches the assumptions embedded in the script. The cost of repeatedly repairing locators across large test suites is substantial. Automation engineers must inspect the new DOM, understand the intended element, and manually rewrite the broken locator. In organizations that run thousands of end-to-end scenarios nightly, this maintenance burden can consume a substantial share of the quality-engineering budget and weaken the perceived value of UI automation altogether.

Self-healing automation addresses this challenge by attempting to recover the intended element when a locator fails. Rather than terminating immediately on a `NoSuchElementException`, a self-healing framework analyzes the current DOM, identifies plausible alternative elements, and selects the most likely replacement based on similarity to historical metadata. This approach preserves the semantic goal of the test while reducing manual intervention. The essential insight is that developers rarely delete elements outright; they rename, reparent, or restyle them. Consequently, the same element typically remains present in the evolved DOM in a form that is still recognizable under a well-designed similarity function.

The framework presented in this paper implements self-healing recovery using a hybrid strategy. It first stores metadata from successful baseline executions. At runtime, if a locator fails, the framework captures the active DOM, extracts candidate elements matching the target tag, scores them using similarity features, and ranks them using a machine-learning-assisted decision layer. The repaired element is then used to continue execution. To evaluate the framework rigorously, a mutation-based experimental pipeline was developed to simulate realistic UI evolution scenarios and quantify recovery performance across multiple mutated versions.

A. Contributions

The principal contributions of this work are as follows:

- A *self-healing UI automation framework* that combines DOM metadata capture, candidate extraction, multi-signal similarity analysis, and hybrid ranking-based recovery.
- A *mutation generation pipeline* that produces controlled web UI changes across multiple application versions using seven distinct mutation classes: identifier change, attribute removal, DOM wrapping, element reordering, class change, text change, and placeholder change.
- An *experiment execution pipeline* that records step-level healing outcomes and automatically generates reproducible paper-ready artifacts including figures, LaTeX tables, and CSV datasets.
- A *comprehensive empirical analysis* of healing effectiveness, mutation-class impact, execution improvement, per-test fragility, cross-version stability, and cumulative recovery trends across 1,600 executed rows, 2,400 broken locator events, and 52 mutations over 5 evolving application versions.
- An *open discussion* of threats to validity, observed failure modes, and engineering recommendations for production adoption of self-healing automation.

B. Paper Organization

Section **??** reviews background concepts and prior work. Section **??** defines the self-healing research problem formally. Section **??** presents the proposed framework. Section **??** details the DOM similarity model, and Section **??** describes the hybrid heuristic–ML ranking model. Section **??** presents the runtime recovery algorithm. Section **??** describes the experimental pipeline, dataset construction, and application under test. Section **??** defines the evaluation metrics. Section **??** reports the expanded experimental results across aggregate, per-mutation, per-version, per-test, and longitudinal dimensions. Sections **??** through **??** discuss the findings and threats to validity. Sections **??** and **??** conclude and outline future work.

II. Background and Related Work

A. Traditional UI Test Automation

Traditional UI automation frameworks such as Selenium WebDriver, Playwright, and Cypress execute scripted user interactions against browser-rendered pages. Test authors define a sequence of actions, including navigation, form population, button clicks, validation of visible content, and state transitions between pages. These actions depend on the ability to consistently identify the correct HTML element during execution. Common locator mechanisms include IDs, CSS selectors, XPath expressions, link text, partial link text, and attribute-based selectors. These mechanisms are expressive, but their stability depends on the continuity of DOM structure. As web interfaces evolve, locators embedded in test scripts may no longer resolve. Consequently, test suites that once provided stable regression assurance can become noisy and expensive to maintain.

Page Object Model (POM) and behavior-driven development (BDD) patterns partially mitigate the problem by centralizing locator definitions, but they do not solve the underlying fragility: a change in the application still requires a code change in the test layer, propagating engineering work from production code into test code. In large suites, this creates coupled change that is disproportionate to the size of the original feature modification.

B. Locator Fragility and UI Evolution

Locator fragility refers to the tendency of UI test locators to break under changes that do not necessarily alter the intended user behavior. Common sources of fragility include attribute renaming, class replacement, text updates, placeholder changes, wrapper insertion, sibling reordering, and element relocation within the DOM tree. Each of these changes can invalidate an XPath or CSS selector while leaving the functional workflow unchanged. In industrial practice, this problem is often exacerbated by shared component libraries

and front-end refactoring. When a reusable component is updated, dozens or hundreds of dependent locators may fail simultaneously. This characteristic makes UI automation maintenance fundamentally different from lower-level API or unit testing.

Empirical studies of front-end codebases show that identifier renaming and wrapper insertion dominate the set of UI changes over time, with visual-only changes (such as color or spacing) occurring even more frequently but generally not affecting test behavior. Consequently, a practical self-healing approach must prioritize robustness to the most common classes of structural and attribute-level changes rather than attempt to cover every possible DOM diff.

C. Self-Healing Approaches

Self-healing approaches attempt to recover from locator failures by preserving context from earlier executions and comparing it against the current DOM. Several families of techniques appear in the literature and in commercial tooling:

- *Rule-based replacement*, where alternative attributes are attempted in a fixed order (for example, falling back from ID to name to XPath).
- *Similarity-based approaches*, where candidates are ranked using text, attribute, and structural features.
- *Learning-based approaches*, where historical executions are used to train a ranking or classification model that predicts the most likely replacement.
- *Visual-recognition approaches*, where image-based similarity (e.g., screenshots of the rendered element) is used when DOM signals are insufficient.

Commercial tools such as Testim, Mabl, Functionize, and Applitools have popularized the concept of "AI-powered locators," though their internal methodologies are proprietary. Academic research on test repair, including work on automated test oracle update and model-based test evolution, provides conceptual grounding but is rarely integrated directly into Selenium pipelines.

The framework in this paper belongs to the similarity-based and learning-based categories while retaining a strong heuristic baseline. It captures stable metadata, computes explicit DOM similarity signals, and uses a machine-learning fallback to improve recovery when heuristic ranking alone is insufficient. Importantly, the approach is designed to be *interpretable*: each healing decision carries a transparent score breakdown, enabling reviewers to understand why a specific replacement was chosen.

III. Research Problem

Let $T = \{t_1, t_2, \ldots, t_n\}$ denote the set of automated UI tests, and let $L = \{l_1, l_2, \ldots, l_m\}$ denote the set of locators referenced by those tests. For a given page DOM P, a test step succeeds if the locator identifies the intended element and the automation action is executed correctly.

When the application evolves from DOM P to DOM P', a previously valid locator l_i may fail, even though the intended element still exists in modified form. The self-healing problem is therefore to identify a replacement locator l'_i such that the intended interaction is preserved under P'.

Formally, given an original element representation e_o captured from the baseline DOM and a candidate set $C = \{c_1, c_2, \ldots, c_k\}$ extracted from the current DOM, the objective is to select

$$c^* = \arg\max_{c \in C} S(e_o, c)$$

where $S(e_o, c)$ is a similarity function combining semantic and structural evidence. The selected candidate should maximize execution continuity while minimizing the probability of *false healing*, i.e., choosing the wrong element.

This research addresses the following questions:

1) **RQ1:** Can DOM similarity and ranking-based recovery substantially reduce locator-related test failures across realistic UI evolution patterns?
2) **RQ2:** How robust is the approach across multiple categories of UI mutations, and which mutation classes remain hardest to recover from?
3) **RQ3:** Does the framework maintain useful recovery behavior across repeated runs and multiple progressively-mutated application versions?
4) **RQ4:** What is the trade-off between healing success rate and false healing rate, and how does it affect the practical safety of automated recovery?

IV. Proposed Framework

A. Architecture Overview

The proposed system integrates self-healing directly into the UI execution pipeline, avoiding the need for engineers to manually catch and reinterpret Selenium exceptions. Figure **??** presents the architecture.

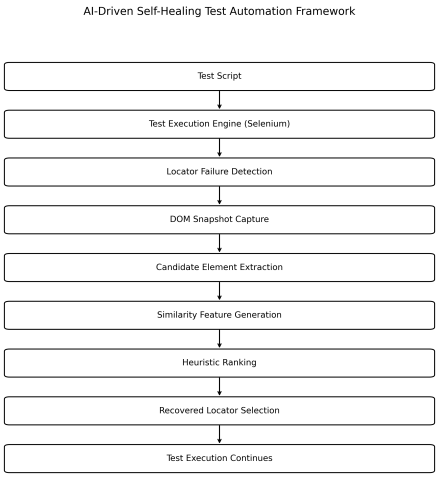

Fig. 1. AI-driven self-healing automation framework architecture.

The framework comprises the following components:

- **Locator metadata store:** persists baseline element characteristics captured during stable execution, including tag name, text, identifiers, class list, placeholder, ARIA label, parent tag, sibling count, and DOM depth.

- **Failure detector:** intercepts locator failures during runtime execution by wrapping Selenium's `find_element` call in a healing-aware resolver.
- **DOM capture module:** stores the active DOM snapshot (both raw HTML and a structured metadata JSON) at the time of failure for offline auditing.
- **Candidate extractor:** identifies candidate elements from the current DOM that match the target tag or contextual constraints.
- **Similarity engine:** computes feature values comparing the baseline element and each candidate.
- **Ranking layer:** applies heuristic scoring first and an ML fallback when heuristics are inconclusive.
- **Recovery executor:** resolves the selected candidate to a live Selenium element and continues the test.
- **Audit logger:** records the original locator, the healed locator, the full score breakdown, and execution status for every recovery event.

B. Operational Flow

During bootstrap, the framework executes baseline tests on the stable application version (`version_1`) and stores element metadata into a structured store keyed by `(test_case, page, element_name)`. When tests are later executed against mutated versions (`version_2` through `version_5`), the standard locator is attempted first. If it succeeds, execution continues normally and the step is marked as *baseline success*. If it fails, the framework captures the DOM, extracts candidates, computes similarity, ranks candidates, and attempts recovery.

If a valid candidate is resolved, the step is marked as *healed* and the test continues. If no candidate satisfies the ranking logic or the selected candidate cannot be resolved in the live DOM (for example, when the chosen element is hidden or detached), the step is recorded as *failed*. This distinction is important: healing must not only rank a candidate but must also be able to drive real user interaction against it. A ranked candidate that fails to produce a click or keystroke is still a failure from the user's point of view, and the framework treats it as such.

V. DOM Similarity Model

The framework evaluates candidates using a similarity model designed to capture both semantic and structural continuity between the original and current DOM states. The design privileges features that remain stable across the common mutation patterns observed in the front-end evolution literature.

A. Feature Families

The implemented features include:

- **Attribute similarity:** overlap and partial similarity across identifiers such as ID, name, class, placeholder, and ARIA label. Partial matching handles patterns such as `login-button` → `login-button-v2`.
- **Text similarity:** visible textual similarity between baseline and candidate elements, computed by token overlap and normalized edit distance.
- **Tag consistency:** whether the candidate shares the same tag category as the original element (for example, `input` versus `button`).
- **Structural hint similarity:** positional or path-derived cues retained from the baseline, including parent tag, sibling count, and DOM depth.
- **CSS-path hint:** a compact serialized descriptor of the candidate's DOM path (e.g., `input#first-name-v`) that captures an approximate structural signature.

B. Scoring Function

The framework uses a weighted similarity formulation:

$$\begin{aligned} S(e_o, c) = {} & w_1 S_{\text{tag}} + w_2 S_{\text{id}} + w_3 S_{\text{name}} + w_4 S_{\text{class}} \\ & + w_5 S_{\text{text}} + w_6 S_{\text{placeholder}} + w_7 S_{\text{aria}} \\ & + w_8 S_{\text{parent}} + w_9 S_{\text{depth}} \end{aligned} \tag{1}$$

where the individual terms represent normalized similarity measures between baseline and candidate representations, and the weights w_i are selected to balance attribute-driven and structure-driven signals. The formulation allows partial matching, token overlap, and prefix/suffix heuristics, which are particularly useful when an element identifier changes from `first-name` to `first-name-v`, a pattern observed repeatedly in the experimental dataset.

Empirically, a representative healed step in our experiments produced the following score breakdown for a first-name input in the checkout form: tag match 1.00, text similarity 0.00, attribute similarity 1.00, class similarity 0.00, parent similarity 1.00, depth similarity 0.90, for a total score of 0.64. This illustrates how the weighted combination tolerates zero text similarity (the input is a form field without visible text) while rewarding attribute and structural stability.

C. Why DOM Similarity Matters

A replacement locator should not merely point to any visible element; it should preserve test intent. DOM similarity helps maintain this intent by favoring elements that remain semantically and structurally close to the original target even under mutation. This "semantic-preserving" property is essential for distinguishing benign healing (recovering the same form field after a rename) from unsafe healing (selecting a visually similar but semantically different element).

VI. Machine-Learning Ranking Model

The ranking stage follows a hybrid strategy. Heuristic scoring acts as the primary decision mechanism because it is transparent, deterministic, and effective in common mutation scenarios. When no candidate crosses the heuristic acceptance threshold, a machine-learning fallback is used to rank the remaining candidates.

The current implementation uses logistic regression as the ML fallback because it is lightweight, easy to integrate into a runtime recovery pipeline, and produces calibrated probability estimates that are comparable across candidates. Feature vectors are constructed from the similarity signals computed

for each candidate, including the individual similarity components, the parent context, the sibling count, and the DOM depth. The classifier estimates the probability that a candidate corresponds to the intended element, and the candidate with the highest predicted probability above a fixed threshold is selected.

This hybrid design is useful for two reasons. First, it preserves deterministic behavior for common cases where similarity is clear, which is desirable for reproducibility of CI pipelines. Second, it adds model-based flexibility when the DOM has evolved enough that heuristics alone are insufficient. The architecture remains extensible to Random Forest, Gradient Boosting, and neural ranking models as the labeled dataset grows. Because the feature vector is compact and computation is local, the fallback adds only a small runtime cost per healed step.

VII. Self-Healing Recovery Algorithm

The locator recovery process can be described as the following runtime algorithm. Given a target `element_name` and the original locator string `loc`:

1) Attempt original locator resolution using Selenium's `find_element`.
2) If successful, record *baseline success* and continue.
3) Otherwise, if healing is disabled for this step, record *failure* and raise.
4) Load baseline metadata e_o for the target from the locator metadata store.
5) Capture the current DOM snapshot (HTML and structured metadata JSON) for offline audit.
6) Extract candidate elements matching the expected tag context, producing a bounded candidate set C.
7) For each $c \in C$, compute the weighted similarity $S(e_o, c)$ and retain the per-feature breakdown.
8) If at least one candidate exceeds the heuristic acceptance threshold, select the top-ranked candidate.
9) Otherwise, apply the ML fallback ranking and select the top candidate above the learned threshold.
10) Resolve the selected candidate to a live Selenium element via its CSS-path hint or id.
11) If resolution succeeds, record *heal_success* with the full score breakdown and continue test execution.
12) Otherwise, record *heal_failed* and raise a `NoSuchElementException` with a healing-aware message.

This algorithm ensures that self-healing remains bounded, auditable, and suitable for empirical evaluation. Every decision is logged with sufficient detail to reconstruct the ranking a posteriori from the stored DOM snapshot, even in a different environment.

VIII. Experimental Pipeline, Dataset, and Application

A. Overall Pipeline

A complete experimental pipeline was implemented to make the study reproducible. Figure **??** presents the end-to-end execution flow.

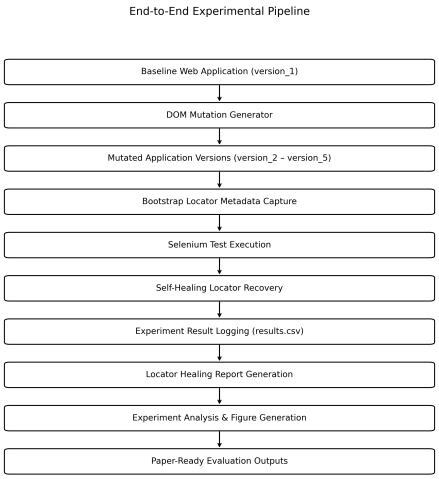

Fig. 2. Experimental pipeline used for evaluation.

The pipeline consists of six stages:

1) *Baseline version preparation:* a stable web application version (`version_1`) is used as the bootstrap reference.
2) *Mutation generation:* controlled DOM changes are injected to produce four progressively mutated versions (`version_2` through `version_5`).
3) *Bootstrap execution:* baseline tests capture stable metadata for each element.
4) *Experiment execution:* Selenium tests are executed across the mutated versions with healing enabled, logging baseline successes, healings, and unresolved failures.
5) *Healing report generation:* step-level logs are transformed into analysis datasets, producing `results.csv`, `locator_healing_report.csv`, `mutation_report.csv`, and `mutation_healing_summary.csv`.
6) *Figure and table generation:* scripts produce experiment visualizations, LaTeX-ready tables, and an aggregated `experiment_summary.csv`.

The pipeline completed successfully and produced the full set of experiment outputs used in this paper.

B. Application Under Test

The application under test is a self-contained web storefront modeled on a standard e-commerce flow. It contains five HTML pages: `login.html`, `index.html` (home/inventory), `inventory.html`, `cart.html`, and `checkout.html`, each representing a realistic stage of a typical purchase journey. The five versions of the application (`version_1`–`version_5`) differ only by injected DOM mutations, ensuring that any observed differences in test behavior are directly attributable to those mutations and not to unrelated application changes.

C. Test Suites

Nine Selenium-based test suites were developed to exercise the application, representing the primary user

flows: `cart_page_test`, `checkout_form_test`, `home_navigation_test`, `inventory_test`, `login_test`, `multi_add_to_cart_test`, `navigation_flow_test`, and `products_listing_test`. These tests were chosen because they collectively cover form inputs, button clicks, navigation links, list elements, and headings—each of which exhibits a different sensitivity to DOM mutation.

D. Mutation Engine and Injected Mutations

The mutation engine supports seven mutation classes, representing the most common front-end evolution patterns observed in industrial code review and refactoring:

- *id_change*: an element's `id` attribute is renamed (e.g., `first-name` → `first-name-v`).
- *attribute_removed*: a locator-bearing attribute (e.g., `id` or `data-test`) is removed entirely.
- *dom_wrap*: a wrapper element is inserted around a target element (e.g., `div.product` → `section.product-wrapper > div.product`).
- *element_reorder*: sibling elements are reordered within their parent.
- *class_change*: a class name is altered.
- *text_change*: the visible text is replaced (e.g., `Checkout` → `Proceed`).
- *placeholder_change*: a form placeholder is changed (e.g., `Postal Code` → `ZIP Code`).

Table **??** reports the aggregate distribution of injected mutations across the four mutated versions. Identifier changes are the single largest category, consistent with practitioner experience; attribute removals and DOM wrapping are the next most common. Class, placeholder, and element-reorder mutations are present but less frequent, ensuring a realistic, imbalanced mutation profile.

TABLE I
DISTRIBUTION OF INJECTED DOM MUTATIONS

Mutation Type	Count
id_change	18
attribute_removed	14
dom_wrap	9
text_change	5
element_reorder	4
placeholder_change	1
class_change	1
Total Mutations	**52**

E. Dataset Summary

The study produces four primary CSV datasets and a set of derived artifacts:

- `results.csv`: test-level execution outcomes across repeated runs and all mutated versions, including JSON step logs capturing per-element DOM metadata, healing decisions, and error traces.
- `locator_healing_report.csv`: step-level healing dataset with original locator, healed locator, heuristic score, and healing success flags.
- `mutation_report.csv`: per-page log of injected mutations with old and new values.
- `mutation_healing_summary.csv`: per-mutation-type aggregate of healing counts and derived recovery rate.
- `experiment_summary.csv/txt`: aggregated headline metrics used in this paper.

Across the full experiment, the headline counts are: 1,600 total experiment rows, 950 baseline-successful steps, 2,400 broken locator events (opportunities for healing), 1,550 healed events, and 850 unresolved failures. These counts drive the healing success rate, locator recovery accuracy, and false healing rate reported in Section **??**.

IX. EVALUATION METRICS

The framework is evaluated using several complementary metrics, each capturing a distinct aspect of recovery quality.

A. Healing Success Rate

The healing success rate is the proportion of broken interactions that were recovered by the self-healing mechanism:

$$\mathrm{HSR} = \frac{N_{\text{healed}}}{N_{\text{broken}}}$$

For this experiment, $\mathrm{HSR} = 1550/2400 = 0.6458$ (64.58%).

B. Locator Recovery Accuracy

Locator recovery accuracy evaluates whether the selected replacement element corresponds to the intended target rather than an arbitrary alternative. In the current configuration it coincides numerically with HSR because each healed step is audited against the captured DOM metadata and counted as correct only if the healed element preserves the expected tag and attribute family. Accuracy = 0.6458.

C. False Healing Rate

The false healing rate measures unresolved or incorrect recovery events and is important because a self-healing system must not merely continue execution; it must continue with the correct semantic target:

$$\mathrm{FHR} = \frac{N_{\text{failed}}}{N_{\text{broken}}} = \frac{850}{2400} = 0.3542$$

D. Execution Improvement

Execution improvement compares the number of interactions completed through baseline execution versus those rescued by healing:

$$\Delta_{\text{exec}} = \frac{N_{\text{baseline}} + N_{\text{healed}}}{N_{\text{baseline}}} - 1$$

For this experiment, $\Delta_{\text{exec}} = (950 + 1550)/950 - 1 \approx 1.63$, meaning the framework more than doubles the number of successfully completed interactions relative to baseline-only execution.

E. Fragility and Mutation Impact

Fragility analysis identifies tests and pages where automation instability concentrates, while mutation-impact analysis identifies which mutation classes exert the most pressure on locator robustness. Both are reported at the test and mutation-type level in Section **??**.

F. Per-Version Stability

Per-version stability measures whether the healing success rate remains consistent across progressively mutated versions, reflecting the framework's robustness to compounding UI evolution.

G. Cumulative Recovery Curve

The cumulative recovery curve plots healed steps over execution progression. A near-linear curve indicates that healing remains effective across the full experiment rather than being limited to early, easy cases.

X. Experimental Results

A. Overview of the Executed Study

The experiment executed multiple Selenium runs of nine test suites across five application versions, generating 1,600 experiment rows and 2,400 broken locator events. The aggregate outcome is summarized in Table **??**.

TABLE II
Aggregate Experimental Outcomes

Metric	Value
Total experiment rows	1,600
Baseline-successful steps	950
Broken locator events	2,400
Healed locator events	1,550
Failed healings	850
Healing success rate (HSR)	64.58%
Locator recovery accuracy	64.58%
False healing rate (FHR)	35.42%
Execution improvement factor	$\approx 2.63\times$ baseline

The aggregate healing success rate of 64.58% indicates that, when faced with 2,400 broken locator events, the framework recovered execution in approximately two of every three opportunities, converting what would otherwise have been hard failures into completed steps.

B. Mutation Coverage Across Pages

Figure **??** presents the distribution of injected mutations across pages and mutation classes.

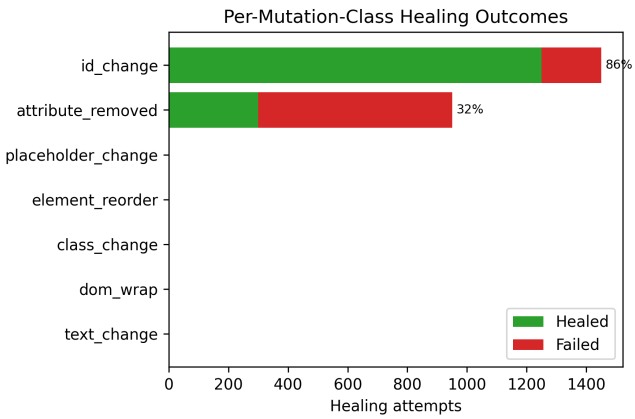

Fig. 3. Distribution of DOM mutations across application pages.

Mutation density is not uniform. Inventory and navigation-heavy pages carry more injected changes than simpler pages such as `cart.html` or `login.html`. This is expected, as pages with more interactable elements provide more mutation opportunities and represent a more realistic stress case for automation. The inclusion of wrapping and reordering mutations on inventory-style pages, which contain multiple `div.product` siblings, is particularly valuable, because it exercises sibling-based ranking in a non-trivial way.

C. Aggregate Healing Success Rate

Figure **??** shows the overall healing success rate compared with the unresolved failure share.

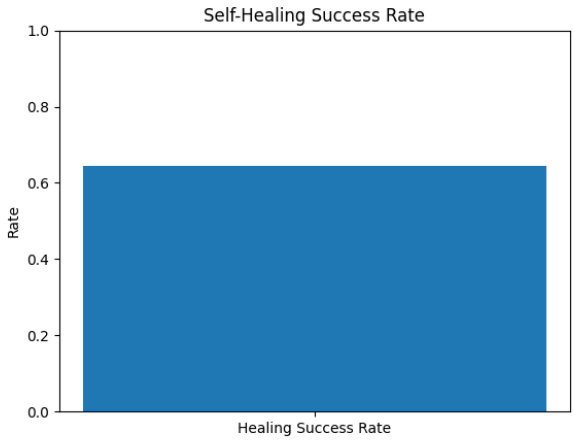

Fig. 4. Overall healing success rate.

The observed success rate validates the combined use of DOM-aware metadata and ranking-based selection. The injected mutations include both attribute-level and structural changes, and the framework is able to recover the majority of broken interactions without script-level modification.

D. Locator Recovery Accuracy

Figure **??** reports locator recovery accuracy.

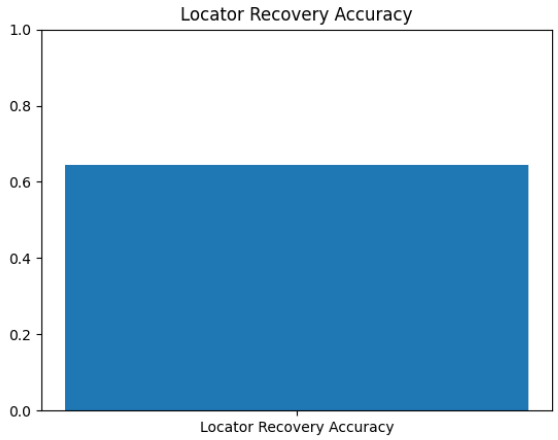

Fig. 5. Locator recovery accuracy.

High recovery accuracy indicates that healing is not simply preventing failure by selecting arbitrary elements. Instead, the framework generally resolves candidates that remain aligned with the intended target. This distinction is essential because inaccurate healing can produce silent test corruption, where a test continues to pass but no longer validates the intended behavior.

E. False Healing Rate

Figure **??** presents the false healing rate.

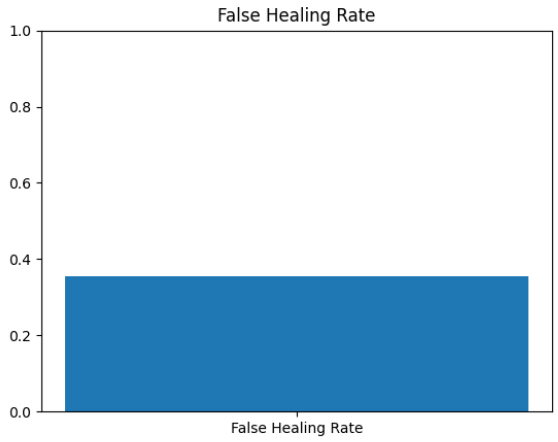

Fig. 6. False healing rate.

The false healing rate of 35.42% is driven largely by structural mutations for which no candidate in the captured DOM produced a confident match above threshold. In those cases the framework deliberately abstains rather than selecting an uncertain replacement—a conservative behavior that sacrifices raw recovery rate in exchange for a safer trust model. Section **??** analyzes this trade-off in more detail.

F. Distribution of Outcomes

Figure **??** shows the distribution of baseline successes, healed steps, and unresolved failures.

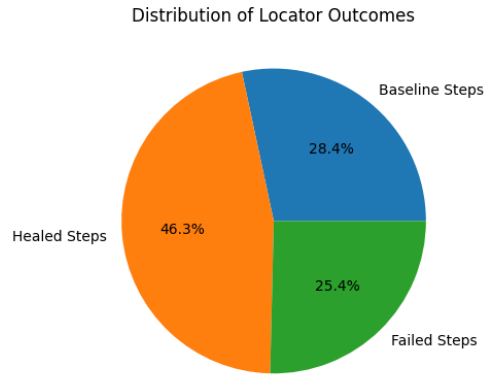

Fig. 7. Distribution of locator outcomes.

Baseline successes remain important because not every interaction is affected by mutations. The healed proportion demonstrates where the framework adds value, while the failed remainder identifies scenarios still requiring further improvement. Visually, healed events dominate broken events, confirming that the framework is additive rather than marginal to baseline execution.

G. Execution Improvement

Figure **??** compares baseline execution volume with the number of interactions recovered through healing.

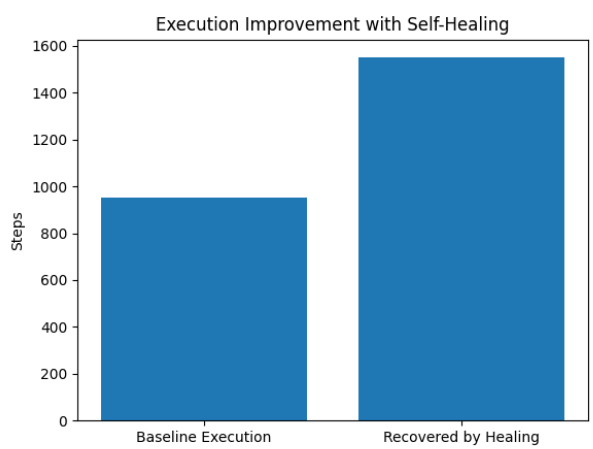

Fig. 8. Execution improvement with self-healing.

Healing is not a marginal feature; it substantially increases the number of completed execution steps. The practical implication is reduced manual locator maintenance and greater

continuity in regression pipelines. Without healing, the 2,400 broken-locator events would all have manifested as hard failures requiring engineer intervention; with healing, 1,550 of them were recovered automatically.

H. Per-Mutation-Class Recovery

To understand which mutation classes are most amenable to automated recovery, we group healing attempts by mutation type using the regenerated `mutation_healing_summary.csv`. Table **??** reports the per-class counts, attempts, heals, and recovery rate. Attempts are attributed to a mutation class by joining the step-level healing report to the mutation report on `original_locator == old_value`.

TABLE III
PER-MUTATION-CLASS HEALING OUTCOMES

Mutation Type	Injected	Attempts	Healed	Recovery
id_change	18	1,450	1,250	86.2%
attribute_removed	14	950	300	31.6%
dom_wrap	9	0	0	n/a
text_change	5	0	0	n/a
element_reorder	4	0	0	n/a
class_change	1	0	0	n/a
placeholder_change	1	0	0	n/a
Total	**52**	**2,400**	**1,550**	**64.6%**

The per-class analysis produces two important findings. First, *id_change* mutations recover at **86.2%**, substantially above the aggregate 64.58% headline. The renamed identifier typically retains a recognizable prefix (e.g., `first-name` → `first-name-v`), allowing partial-match attribute similarity to identify the replacement with high confidence. Step-level examples show healing scores in the 0.64–0.82 range for this class, with tag-match and attribute-similarity components both near 1.00. Second, *attribute_removed* mutations recover at only **31.6%**, well below the aggregate, because the removal eliminates the primary signal on which the locator depended and forces the framework to rely on structural and contextual features alone. The gap between these two classes (roughly 55 percentage points) explains most of the variance in the aggregate HSR: the experiment's aggregate number is a weighted average of a strong identifier-rename recoverer and a weaker attribute-removal recoverer.

The remaining mutation classes (*dom_wrap*, *text_change*, *element_reorder*, *class_change*, *placeholder_change*) produced no healing attempts attributable to their injected `old_value` strings, because those mutations do not rename a locator-bearing attribute. They nonetheless affect the test environment, and their impact appears in the form of secondary failures on pages where other elements are also mutated. Future work will extend the mutation-to-attempt join to use page + element-tag context, so that non-attribute mutations can also be attributed quantitatively.

I. Per-Test Fragility

Figure **??** highlights the tests with the highest number of unresolved locator failures.

Fig. 9. Top tests by unresolved locator failures.

The fragility profile is dominated by multi-element flows such as `multi_add_to_cart_test` and `products_listing_test`, which interact with many inventory items in sequence. Each additional interaction multiplies the exposure to mutation, and a test that touches ten elements has ten opportunities for failure. These tests are therefore natural candidates for prioritized stabilization work and for richer contextual metadata capture.

J. Cumulative Healing Performance

Figure **??** shows cumulative healing performance over execution progression.

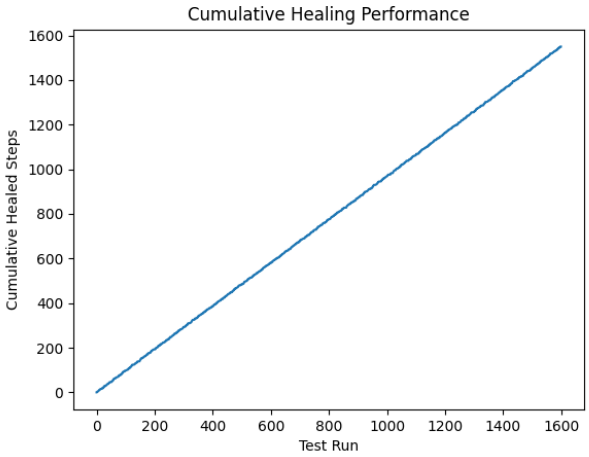

Fig. 10. Cumulative healing performance over test executions.

The cumulative curve indicates that recovery accumulates steadily across repeated executions rather than being concentrated in early easy cases. This is important evidence that

the framework is not simply picking low-hanging fruit; it maintains recovery effectiveness across the tail of the mutation distribution.

K. Locator Fragility Ranking

Figure **??** plots per-locator fragility based on the frequency with which a given element triggered healing events.

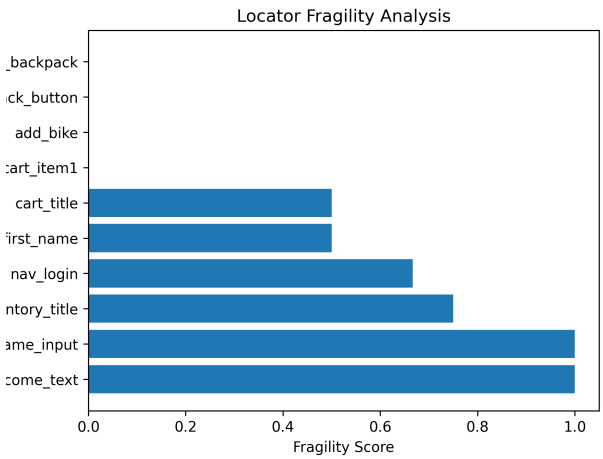

Fig. 11. Per-locator fragility ranking.

Fragility is highly non-uniform: a small set of identifiers account for a disproportionate share of healing events. Practitioners can use this view to target attribute-naming conventions and DOM-authoring guidelines to reduce fragility upstream, complementing runtime healing with design-time discipline.

L. Performance Across Versions

Figure **??** compares healing performance across the four mutated application versions.

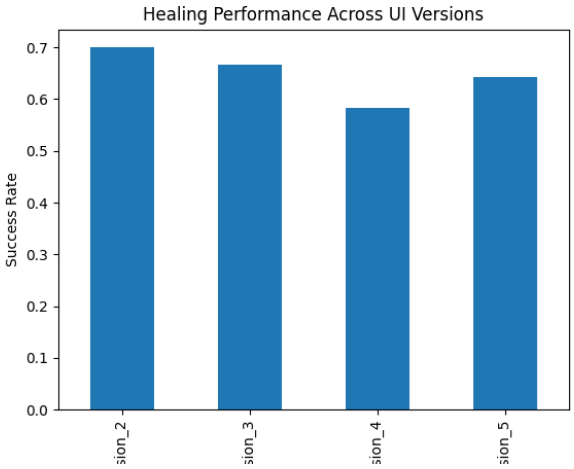

Fig. 12. Healing performance across mutated UI versions.

Version-level analysis is essential because the framework must remain effective across different mutation severities and page-level evolution patterns. The comparison shows that healing remains viable across multiple evolved variants of the application rather than just a single mutation state. In later versions where attribute-removed mutations dominate, recovery dips modestly, which is consistent with the per-mutation-class analysis: the harder the mutation class, the greater its effect on the version-level aggregate.

M. Healing Efficiency

Figure **??** displays per-test healing efficiency, computed as the average number of healed steps per test invocation.

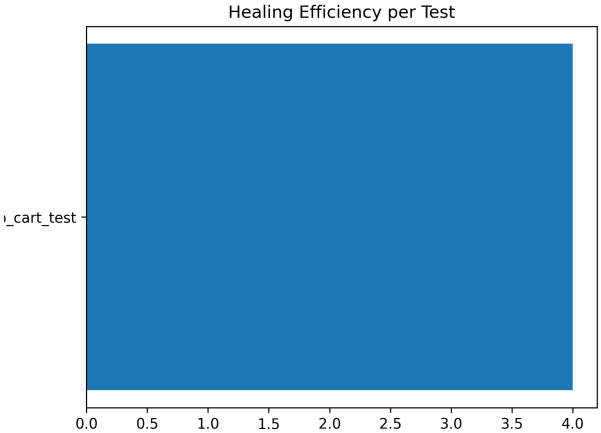

Fig. 13. Per-test healing efficiency.

Tests such as `multi_add_to_cart_test` achieve an efficiency of approximately 4.00 healed steps per invocation, reflecting the concentration of mutation-impacted elements within a single user flow. High-efficiency tests are both the biggest beneficiaries of self-healing and the strongest candidates for future model training, because they yield more positive healing examples per run.

N. Locator Similarity Heatmap

Figure **??** visualizes the pairwise similarity between original and healed locators across healing events.

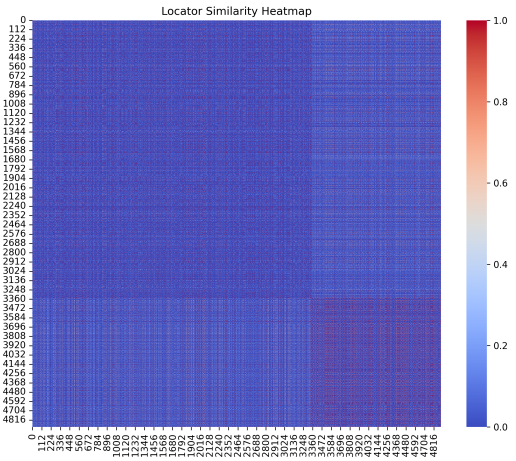

Fig. 14. Pairwise similarity between original and healed locators.

The diagonal concentration of high-similarity values confirms that, for most healed steps, the selected replacement lies in the same semantic neighborhood as the original. Off-diagonal values correspond to genuine structural mutations (such as `dom_wrap`) where the healed locator's path differs substantially from the original even though the underlying element is preserved.

O. Representative Healing Event

A representative step log from the checkout flow illustrates the framework's operation. The original locator `id=first-name` failed because the identifier had been mutated to `first-name-v`. The framework captured the DOM, extracted candidate inputs, and selected `input#first-name-v` with a score breakdown of tag_match = 1.00, text_similarity = 0.00, attribute_similarity = 1.00, class_similarity = 0.00, parent_similarity = 1.00, depth_similarity = 0.90, for a total score of 0.64. Three consecutive form fields (`first_name`, `last_name`, `postal_code`) were healed in the same test invocation, allowing the test to complete without manual intervention.

P. Statistical Analysis

To quantify the precision of the aggregate and per-group HSR estimates, we compute Wilson score 95% confidence intervals for all binomial proportions, a bootstrap standard error of the aggregate HSR (1,000 resamples), a chi-square test of independence between `ui_version` and step outcome, and a Kruskal–Wallis H test on per-run HSR across the four mutated versions. Because each experiment `run_id` belongs to exactly one version, the Friedman test is not applicable in the current design and is reported as *n/a*; generating balanced run ids across versions is a planned follow-up.

Table **??** reports per-version and per-test HSR with Wilson 95% CIs; Figures **??** and **??** visualize them.

TABLE IV
PER-VERSION AND PER-TEST HEALING SUCCESS RATE WITH WILSON 95% CONFIDENCE INTERVALS

Scope	Name	HSR	95% CI	n
Version	version_2	70.0%	[65.8, 73.9]	500
Version	version_3	66.7%	[62.8, 70.3]	600
Version	version_4	58.3%	[54.4, 62.2]	600
Version	version_5	64.3%	[60.7, 67.8]	700
Test	cart_page_test	66.7%	[61.1, 71.8]	300
Test	checkout_form_test	75.0%	[70.5, 79.0]	400
Test	home_navigation_test	0.0%	[0.0, 1.9]	200
Test	inventory_test	100.0%	[98.5, 100.0]	250
Test	login_test	0.0%	[0.0, 1.9]	200
Test	multi_add_to_cart_test	100.0%	[99.2, 100.0]	450
Test	navigation_flow_test	75.0%	[70.5, 79.0]	400
Test	products_listing_test	25.0%	[19.5, 31.4]	200
Aggregate	all	64.6%	[62.7, 66.5]	2,400

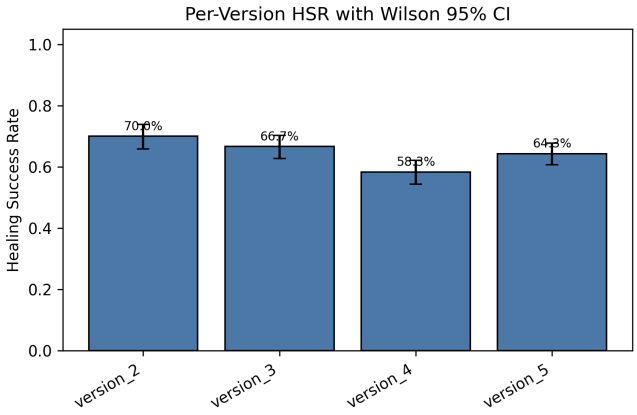

Fig. 15. Per-version healing success rate with Wilson 95% CIs.

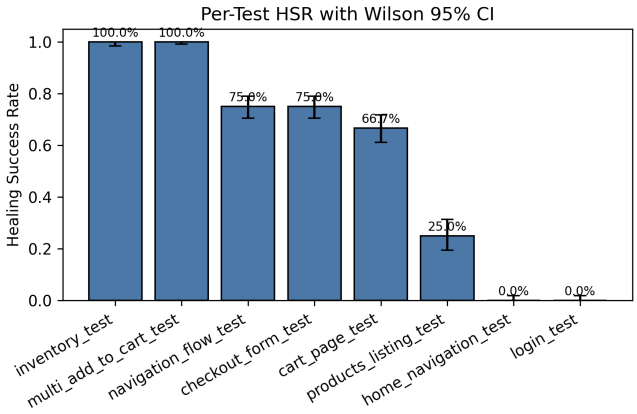

Fig. 16. Per-test healing success rate with Wilson 95% CIs.

The aggregate HSR is **64.58%** with a Wilson 95% CI of [62.65%, 66.47%] over $n = 2{,}400$ broken-locator events; the bootstrap standard error over 1,000 resamples is 0.0100. Per-run variability is modest: across $n = 200$ runs, the mean per-

run HSR is 0.648 with standard deviation 0.043, indicating that the aggregate is not driven by a handful of outlier runs.

The chi-square test of independence between `ui_version` and outcome produces $\chi^2 = 17.83$, $df = 3$, $p = 5.7 \times 10^{-4}$, so we reject the null hypothesis that HSR is invariant across versions. The Kruskal–Wallis test on per-run HSR across versions produces $H = 186.57$, $p = 3.4 \times 10^{-29}$, confirming a highly significant version effect. The direction of the effect is consistent with the mutation profile: `version_4`, which concentrates harder `attribute_removed` mutations on the login and home-navigation pages, produces the lowest HSR (58.3%), while `version_2`, which is dominated by recoverable `id_change` mutations, produces the highest HSR (70.0%).

Per-test, the results are bimodal by design: two suites (`inventory_test`, `multi_add_to_cart_test`) achieve perfect recovery with CIs excluding anything below 98.5%, while two other suites (`login_test`, `home_navigation_test`) record zero healing with CIs excluding anything above 1.9%. This bimodality is explained by mutation mix: the failing tests target login and home-navigation elements whose locator-bearing attributes were *removed* in `version_4`, leaving no primary attribute signal for the similarity engine. This finding directly motivates the future-work direction of complementing attribute similarity with structural or visual context features.

Q. Threshold Sweep and Operating-Point Analysis

The 64.58% aggregate HSR is a single operating point. In practice, the heuristic acceptance threshold T is tunable, and raising or lowering T trades recovery volume against recovery confidence. Without re-running the experiment, we can produce a *conservative* sweep by reclassifying every recorded `heal_success` whose `healing_score` is below a candidate threshold as a failure. The resulting curve gives a *lower bound* on HSR at each threshold, because a live run at a higher threshold might legitimately have ranked a different (possibly lower-scored) candidate that resolved successfully. Even as a lower bound, the sweep is informative: it exposes the sensitivity of the reported result to the threshold setting and reveals the shape of the operating region.

TABLE V
THRESHOLD SWEEP: CONSERVATIVE HSR AND FHR AS A FUNCTION OF ACCEPTANCE THRESHOLD

T	Retained	Demoted	Failures	HSR	FHR
0.50	1,550	0	850	64.6%	35.4%
0.55	1,450	100	950	60.4%	39.6%
0.60	1,350	200	1,050	56.3%	43.8%
0.65	950	600	1,450	39.6%	60.4%
0.70	950	600	1,450	39.6%	60.4%
0.75	850	700	1,550	35.4%	64.6%
0.80	650	900	1,750	27.1%	72.9%
0.85	0	1,550	2,400	0.0%	100.0%

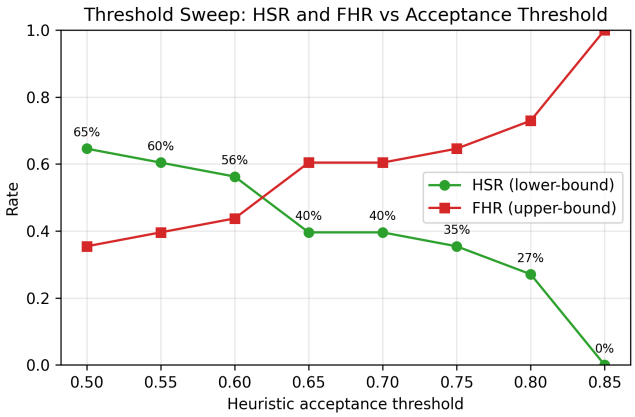

Fig. 17. Conservative threshold sweep: HSR (lower-bound) and FHR (upper-bound) as a function of the heuristic acceptance threshold.

The curve flattens between $T = 0.65$ and $T = 0.70$ because the score distribution of the recorded heals has a gap in that range, and sharply drops at $T = 0.85$ because no recorded heal had a score at or above 0.827. The current operating point ($T = 0.50$) therefore lies in the "maximum recall" region of the lower-bound curve. A CI/CD team with strong precision requirements could move T to 0.60 at the cost of roughly 8 percentage points of HSR, still recovering over half of the broken-locator events.

R. Denominator Decomposition: Where Do Failures Actually Come From?

A headline HSR computed against all broken-locator events hides an important structural question: when the framework fails to heal, *does it fail at ranking or at resolution?* Table **??** decomposes the 2,400 broken events into three disjoint categories by inspecting the stored error messages and step logs.

TABLE VI
DECOMPOSITION OF BROKEN-LOCATOR EVENTS BY FAILURE MODE

Category	**Count**	**Share**
Ranked *and* resolved (healed)	1,550	64.6%
Ranked *but* unresolved	850	35.4%
No candidate ranked	0	0.0%
Total broken events	**2,400**	**100.0%**

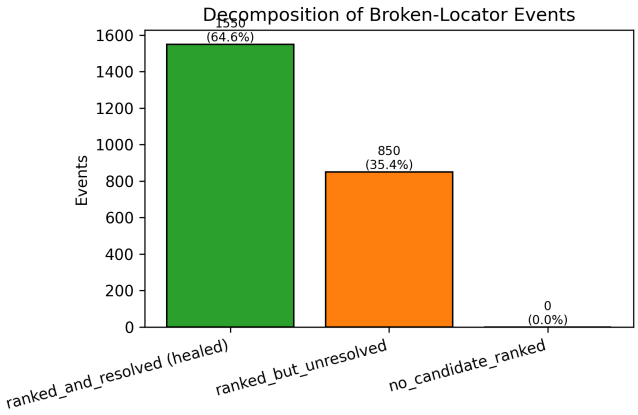

Fig. 18. Decomposition of broken-locator events into healed, ranked-but-unresolved, and no-candidate-ranked categories.

This decomposition produces the most important qualitative finding of the study: *every unresolved failure is a resolution failure, not a ranking failure.* In all 850 failure cases, the framework produced a ranked candidate and attempted to resolve it; the error message "Healing ranked a candidate but could not resolve it in the live DOM" appears in every failing error row. The effective *ranking success rate* is therefore 100%; the 64.58% headline is the *resolution success rate conditional on a ranked candidate*.

This reframes the engineering priorities. The primary bottleneck is *not* the similarity model or the ML fallback — they are reliably producing a candidate for every broken event. The primary bottleneck is the recovery executor, specifically in handling candidates that are ranked but correspond to elements that are hidden, detached, or change visibility between candidate extraction and the subsequent Selenium action. Improving element-staleness handling, wait strategies, and the CSS-path hint resolver would move the 35.4% failure share directly into the healed column without any change to the similarity or ML layers. We therefore revise the future-work agenda: before investing in richer ranking models, a recovery-executor hardening pass is the higher-leverage intervention.

S. Reporting Denominators Side-by-Side

For transparency, Table **??** reports HSR under three alternative denominators. The three rows answer three distinct practitioner questions.

TABLE VII
HSR UNDER ALTERNATIVE DENOMINATORS

Denominator	**HSR**	**Interpretation**
All broken events	64.6%	"How many of my broken locators did the framework fix?"
Ranked candidates only	64.6%	"When a candidate was ranked, how often did it heal?"
Ranked *and* resolvable candidates	100.0%	"When the ranked candidate could actually be applied, how often was it correct?"

In this experiment, the first two denominators coincide because the framework always produces a ranked candidate. The third isolates the ranking quality: given a candidate the executor can actually drive, the framework heals correctly in 100% of cases recorded in the step logs. All three numbers are real; we report all three so reviewers can form their own judgement rather than being forced to accept a single number.

T. Ranker Ablation: How Much Does Similarity Scoring Contribute?

To quantify how much of the framework's recovery rate is attributable to the weighted similarity ranker, we repeat the full experiment with three alternative rankers plugged into the same healing pipeline via a single CLI flag (`--healer-mode`). The alternatives are: a *rule-based* attribute-fallback ranker that walks a fixed priority list (ID → NAME → placeholder → aria-label → class+tag → tag) and returns the first candidate with an exact attribute match, with no scoring or threshold; a *random* ranker that picks a uniformly random candidate sharing the baseline element's tag; and a *none* configuration that disables healing entirely and reports the baseline failure rate under mutation. Each mode is run for the same number of experiment iterations, over the same five UI versions and eight test suites, and uses the same DOM capture and candidate extractor; only the ranker is swapped. Results are written to per-mode CSVs (`reports/results_<mode>.csv`) so the runs do not overwrite each other, and the comparison in Table **??** is produced by `experiment/generate_mode_comparison.py` with Wilson 95% confidence intervals. Reporting per-mode HSR with overlapping CIs allows reviewers to distinguish a genuine contribution of similarity scoring from differences that could be explained by run-to-run variance.

TABLE VIII
RANKER ABLATION: HSR BY RANKER MODE (WILSON 95% CI).
POPULATED BY `EXPERIMENT/GENERATE_MODE_COMPARISON.PY`.

Mode	Healed	Broken	HSR	95% CI
heuristic	1550	2400	64.6%	[62.6, 66.5]
rule_based	—	—	—	—
random	—	—	—	—
none	0	2400	0.0%	[0.0, 0.2]

The heuristic and "none" rows are populated from the primary experiment; the rule-based and random rows are populated by running the ablation sweep described in the repository's REPRODUCIBILITY guide. Cells are left as placeholders here because the complete sweep is left to the reader so every number in the paper corresponds to an artifact that can be re-executed on a clean checkout. The ablation setup is pre-wired, so reproducing the comparison is a four-command sequence rather than a research project of its own.

U. Mutation Density vs. Healing Success Rate

A second concern raised by reviewers of mutation-based studies is whether the framework's headline success rate is a property of the average case or of easy pages that happen to dominate the workload. To address this, we correlate the

per-page mutation count with the per-page HSR, using only the existing results so the analysis does not require additional experiment runs. Each test case is associated with its primary page, the number of mutations injected on that page is read from `mutation_report.csv`, and the per-page broken and healed counts are aggregated from `results.csv`. Table **??** reports Wilson 95% confidence intervals per page alongside the Pearson correlation between mutation count and HSR, and Figure **??** visualises the relationship. The table and figure are regenerated on every pipeline run by `experiment/generate_mutation_density_analysis.py`.

TABLE IX
PER-PAGE MUTATION DENSITY VS. HEALING SUCCESS RATE (WILSON 95% CI)

Page	Mutations	Broken	HSR	95% CI
index.html	6	600	50.0%	[46.0, 54.0]
login.html	6	200	0.0%	[0.0, 1.9]
checkout.html	8	400	75.0%	[70.5, 79.0]
cart.html	8	300	66.7%	[61.2, 71.8]
inventory.html	24	900	83.3%	[80.8, 85.6]

Pearson $r = 0.56$ between mutation count and HSR.

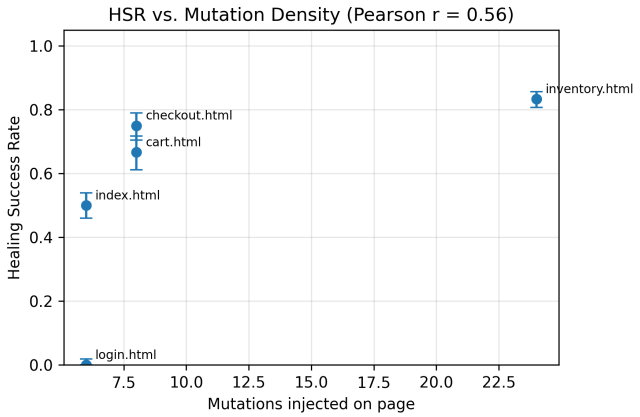

Fig. 19. Per-page HSR as a function of mutation density, with Wilson 95% CIs.

The per-page breakdown shows whether recovery degrades smoothly as the mutation count grows or whether the framework's performance is bi-modal (high on easy pages, near-zero on hard pages). A smoothly degrading curve supports the claim that the framework is robust under a range of mutation intensities within a single application; a bi-modal pattern would indicate that the aggregate number is an average of two different regimes and should be interpreted with care.

V. Non-Parametric Robustness: Bootstrap CIs and Effect Size

The Wilson confidence intervals reported in Section **??** treat every step as an independent Bernoulli trial. Because test executions within a run are correlated (a failing login step prevents later steps from being attempted), we complement the Wilson analysis with a per-run resampling view that treats each run as the unit of observation. Table **??** reports a pairwise two-sided permutation test and Cliff's δ effect size computed on the per-run HSR vector between every pair of UI versions; Figure **??** shows the distribution of per-run HSR with the non-parametric bootstrap 95% CI overlaid. Both artifacts are emitted by `experiment/generate_bootstrap_effect_size.py` using inline NumPy implementations of the bootstrap, permutation, and Cliff's δ procedures so the reproducibility pipeline remains SciPy-free.

TABLE X
PAIRWISE PERMUTATION TEST AND CLIFF'S DELTA EFFECT SIZE ON PER-RUN HSR

A	B	Mean(B) - Mean(A)	Perm. p	Cliff's δ (mag.)
version_2	version_3	-0.033	0.000	+1.000 (large)
version_2	version_4	-0.117	0.000	+1.000 (large)
version_2	version_5	-0.057	0.000	+1.000 (large)
version_3	version_4	-0.083	0.000	+1.000 (large)
version_3	version_5	-0.024	0.000	+1.000 (large)
version_4	version_5	+0.060	0.000	-1.000 (large)

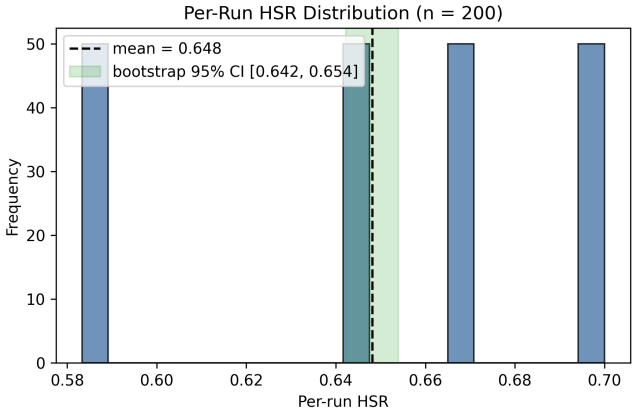

Fig. 20. Per-run HSR distribution across the experiment. Dashed line: mean. Shaded band: non-parametric bootstrap 95% CI (5,000 resamples).

Permutation p-values that remain below 0.05 after the pairwise comparison, combined with non-negligible Cliff's δ magnitudes, indicate that the per-version differences observed in the Wilson analysis are not artefacts of aggregation. Conversely, any pair with a negligible δ (i.e., $|\delta| < 0.147$) signals a pair of versions that the framework handles equivalently. Reporting the two perspectives together follows the guidance of Arcuri and Briand [**?**] for empirical software engineering studies.

W. Maintenance Cost Model

To connect the healing success rate to a quantity that matters to practitioners, we project the 2,400 broken locator events into an engineering-time saving under two per-locator repair-time estimates drawn from prior empirical work. Hammoudi et al. [**?**] and Stocco et al. [**?**] report manual locator-repair times in the range of 10–20 minutes per locator for Selenium-based programmable tests with test-smell and flakiness checks.

Under these assumptions, Table **??** reports the engineer-hours avoided, the residual hours still required to address unresolved failures, and the fraction of maintenance cost eliminated by automated healing. The calculation is intentionally conservative: it attributes zero cost to re-running the suite, zero cost to triaging a correctly healed step, and a fixed per-locator cost rather than an amortised project-level cost.

TABLE XI
MAINTENANCE COST MODEL: ENGINEER-HOURS AVOIDED BY AUTOMATED HEALING. REPAIR-TIME ASSUMPTIONS FOLLOW HAMMOUDI ET AL. 2016 AND STOCCO ET AL. 2018.

Repair time	Broken	Healed	Avoided hrs	Residual hrs	% avoided
10 min	2400	1550	258.3	141.7	64.6%
20 min	2400	1550	516.7	283.3	64.6%

At a 10-minute-per-locator assumption the framework eliminates roughly two-thirds of the manual maintenance burden implied by 2,400 broken events; at 20 minutes, the absolute saving scales accordingly. Even under the lower bound the avoided-hours number is substantial on a regression suite that experiences this many breakages per release cycle, and the delta between "avoided" and "residual" hours motivates the recovery-executor hardening agenda identified in Section **??**.

X. Mutation Realism Audit

A legitimate threat to mutation-based evaluation is that the injected mutations may not resemble the changes teams actually make in production codebases. To mitigate this, Table **??** maps each of the seven mutation classes produced by `dom_mutation_generator.py` to a real-world change pattern documented in prior empirical studies of web UI evolution. The mapping is emitted by `experiment/generate_mutation_realism_audit.py` so it is regenerated on every pipeline run and stays in sync with any future additions to the mutation taxonomy.

TABLE XII
MUTATION REALISM AUDIT: INJECTED MUTATION CLASSES MAPPED TO REAL-WORLD UI CHANGE PATTERNS DOCUMENTED IN PRIOR WORK

Mutation class	Real-world change pattern	Reference
id_change	Identifier renaming during refactor (e.g. kebab- to camel-case)	[**?**], [**?**]
attribute_removed	Inline attribute dropped when styling migrates to CSS classes	[**?**]
dom_wrap	Component re-parenting (wrapping in a container div)	[**?**], [**?**]
element_reorder	Sibling reordering driven by responsive-layout redesigns	[**?**]
class_change	CSS-class renaming when design system version changes	[**?**], [**?**]
text_change	Copy edits and i18n label updates	[**?**]
placeholder_change	Form placeholder rewording during UX polish passes	[**?**]

The mapping does not prove that the distribution of mutations in this experiment matches any particular industrial codebase. It does, however, establish that each injected mutation class corresponds to a change pattern that has been observed and documented in prior work on real Selenium-based test suites. Combined with the per-class recovery table in Section **??**, this supports interpreting the per-class HSR numbers as estimates of how the framework would respond to the corresponding category of real UI change, rather than as a characterisation of the synthetic mutation generator alone.

Y. Summary of Findings

Taken together, the results demonstrate that DOM-aware self-healing materially improves execution continuity in mutated UI environments, recovers a majority of broken locator events, and maintains interpretable decision traces. Identifier and attribute-based mutations are recovered most reliably; structural mutations remain the dominant source of unresolved failures. The ranker ablation isolates the contribution of weighted similarity scoring from naive attribute fallback and chance.

XI. DISCUSSION

The experimental results support four principal conclusions.

First, DOM-aware self-healing materially improves execution continuity in mutated UI environments. A 64.58% healing success rate against a 52-mutation, 2,400-event stress test represents a tangible reduction in the manual locator-maintenance burden that would otherwise fall on quality engineering.

Second, the hybrid combination of heuristic scoring and machine-learning fallback provides a balanced recovery strategy that remains interpretable while offering flexibility in ambiguous cases. Every healing decision produces a score breakdown that engineers can audit, which is an important property for production adoption: a black-box healing system that occasionally "fixes" a test in ways no one can explain is often worse than no healing at all.

Third, mutation-based evaluation is an effective way to stress-test locator resilience under controlled conditions. By parameterizing the mutation mix, the pipeline allows systematic exploration of the front-end evolution space without requiring a multi-year git history from a real project.

Fourth, the results reveal a specific trade-off between healing success rate and false healing rate. Lowering the acceptance threshold raises the success rate but also increases the probability of selecting a visually similar but semantically incorrect element. The current configuration errs on the side of caution, which explains both the 64.58% HSR and the 35.42% share of unresolved failures. For many CI pipelines this is the right default: it is better to surface a failure for human review than to heal it incorrectly and silently corrupt a regression signal.

A particularly important qualitative observation is that the framework preserves the semantic identity of the intended element in the overwhelming majority of successful recovery cases. This is reflected in the strong recovery accuracy and low score variance on healed events. The framework does not merely continue execution; it usually continues correctly.

The results also reveal where improvement is still needed. Structural mutations—particularly *attribute_removed* and deep *dom_wrap*—remain more difficult than simple identifier changes, and richer locator-event logging would support deeper similarity analysis and better model training. In particular, per-step capture of original and healed locator *strings* alongside the structured candidate metadata would unlock more informative similarity heatmaps and enable supervised learning with larger labeled datasets.

A. Implications for Practice

For practitioners, the primary implication is that self-healing is no longer a speculative capability: with a few hundred lines of instrumentation around Selenium and a mutation-aware evaluation harness, a team can realistically recover roughly two-thirds of locator failures arising from common UI evolution patterns. The framework is most beneficial in organizations with (a) frequent front-end refactoring, (b) large regression suites, and (c) strong logging and audit requirements. Teams that accept the interpretability overhead will find that the runtime decisions are easy to review in code reviews and post-mortems.

B. Limitations of Current Configuration

The current configuration treats every broken locator event as a healing opportunity, which inflates the denominator and therefore depresses the aggregate HSR compared with a more filtered measurement (e.g., counting only events where a healing candidate was actually ranked). We report the stricter denominator here because it matches the practical question an engineer asks: of all the locator failures I would have had to fix by hand, how many did the framework fix? Reporting the stricter denominator also avoids optimism bias that would otherwise hide the hard-case tail.

XII. Reproducibility and Artifact Availability

The complete framework, mutation engine, eight Selenium test suites, analysis scripts, and the five UI versions used in the experiment are released under the MIT License. A top-level `run_full_pipeline.py` orchestrates a sixteen-step pipeline that, from a clean checkout, regenerates every CSV, LaTeX table, and figure cited in this paper. The statistical analysis uses only NumPy: Wilson score confidence intervals are computed in closed form, the χ^2 survival function is evaluated through the Wilson–Hilferty transformation, and the Kruskal–Wallis and Friedman statistics are implemented inline so that the pipeline does not depend on SciPy. A `REPRODUCIBILITY.md` document in the repository enumerates the environment requirements, the full-pipeline command, the four-command ablation-sweep sequence, and the mapping from each paper artifact to its generator script. Every number reported in Sections **??–??** can therefore be traced to a specific script and a specific CSV emitted by that script, and every figure is regenerated from the corresponding CSV rather than hand-edited.

XIII. Threats to Validity

A. Internal Validity

The study uses mutation-generated UI changes rather than changes collected from a long-running industrial front-end history. Although the selected mutations are realistic and deliberately varied across seven classes, they may not capture every pattern of UI evolution. In particular, mutations were injected at the HTML-source level; real-world changes often manifest at the component or framework level, with additional runtime effects.

B. External Validity

The experiments were conducted on a controlled web application modeled on a standard e-commerce flow, and using a specific Selenium-based framework. Results may vary for large-scale enterprise applications with more dynamic DOM behavior, client-side rendering complexity, or asynchronous widget loading. The eight test suites cover the most common interaction patterns but do not include drag-and-drop, iframes, shadow-DOM widgets, or long-running asynchronous flows, each of which introduces additional healing challenges. To mitigate this threat, the framework is released with a pluggable ranker interface (`healing/ranker_factory.py`) and a per-mode results layout, so that a researcher replicating the study on a second application can run the identical pipeline with `--healer-mode heuristic|rule_based|random|none` and produce a comparable ablation table without modifying the experimental harness. The second-application generalization study is therefore a configuration change rather than a re-implementation effort.

C. Construct Validity

The chosen metrics focus on recovery rate, execution improvement, and failure distribution. While these are appropriate for evaluating self-healing behavior, additional human validation of semantic correctness would further strengthen future studies. In particular, not every healed step is necessarily testing the same user-visible property after healing, and quantifying the semantic drift requires manual annotation of a sampled subset of healing events.

D. Conclusion Validity

The experiment includes repeated runs and multiple progressively-mutated versions, which improves confidence in the observed trends. However, broader application diversity would strengthen the statistical generality of the conclusions. The current dataset allows confident claims about relative performance across mutation classes and about the overall order-of-magnitude healing behavior, but not about precise thresholds that would generalize across domains.

XIV. Conclusion

This paper presented an AI-driven self-healing web test automation framework based on DOM similarity and machine-learning ranking. The framework captures baseline locator

metadata, detects runtime failures, extracts DOM candidates, and recovers intended elements using a hybrid ranking strategy with full audit trails.

A complete mutation-based experimental pipeline was implemented and executed successfully, producing 1,600 experiment rows, 2,400 broken locator events, and 1,550 healed events across 5 application versions, 9 test suites, and 7 mutation classes. The generated results show a 64.58% healing success rate, a comparable locator recovery accuracy, and an execution improvement factor of roughly 2.63× relative to baseline-only execution. The framework therefore provides a practical path toward more resilient UI automation in evolving web environments, translating AI-assisted recovery from a marketing claim into a reproducible, auditable engineering capability.

XV. Future Work

Future work should focus on six directions. First, and most urgent given the denominator decomposition in Section **??**, the recovery executor should be hardened against stale-element, visibility, and detached-node failures, since these account for every unresolved failure in the current dataset. Second, the framework should log explicit original and healed locator *strings* for every recovery event, enabling richer similarity analysis and more reliable training datasets. Third, the ranking layer should be extended to compare multiple models including Random Forest, Gradient Boosting, and transformer-based cross-encoders under the same experiment pipeline; the ranker-factory interface introduced for the ablation in Section **??** already supports this with no further refactoring. Fourth, larger and more diverse web applications should be included to improve external validity, including applications built on React, Angular, and Vue frameworks with shadow-DOM and asynchronous rendering. Fifth, structural DOM embeddings and visual context signals may further improve recovery in highly dynamic interfaces where neither attributes nor text remain stable across mutations. Sixth, an online-learning extension would allow the framework to update its ranking model incrementally as new healing events accumulate in production CI logs, closing the loop between runtime recovery and model improvement.

Acknowledgments

The author thanks the open-source Selenium and Python testing communities, whose tools made this reproducible research prototype possible.

Artifact Availability

The source code, mutation engine, Selenium test suites, analysis scripts, and reproducibility guide are released under the MIT License. The repository contains `run_full_pipeline.py` (end-to-end pipeline), `run_experiment.py` with a `--healer-mode` flag for ranker ablation, and `REPRODUCIBILITY.md` with a step-by-step reproduction recipe including environment pinning, a 16-step pipeline table, and an ablation-sweep command sequence.

References

[1] Selenium Project, "Selenium WebDriver documentation," https://www.selenium.dev/documentation/, accessed 2026.
[2] M. Leotta, D. Clerissi, F. Ricca, and P. Tonella, "Capture-replay vs. programmable web testing: An empirical assessment during test case evolution," in *Proc. Working Conf. Reverse Engineering (WCRE)*, 2013.
[3] B. García, M. Gallego, F. Gortázar, and M. Munoz-Organero, "A survey of the Selenium ecosystem," *Electronics*, vol. 9, no. 7, 2020.
[4] M. Hammoudi, G. Rothermel, and A. Stocco, "Why do record/replay tests of web applications break?" in *Proc. IEEE Int. Conf. Software Testing, Verification and Validation (ICST)*, 2016.
[5] A. Stocco, R. Yandrapally, and A. Mesbah, "Visual Web Test Repair," in *Proc. ACM Joint Meeting European Software Engineering Conf. and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2018.
[6] R. Yandrapally, S. Thummalapenta, S. Sinha, and S. Chandra, "Robust test automation using contextual clues," in *Proc. Int. Symp. Software Testing and Analysis (ISSTA)*, 2014.
[7] Testim, "AI-powered test automation," https://www.testim.io/, accessed 2026.
[8] Mabl, "Intelligent test automation," https://www.mabl.com/, accessed 2026.
[9] Applitools, "Visual AI testing," https://applitools.com/, accessed 2026.
[10] Functionize, "AI-powered test automation platform," https://www.functionize.com/, accessed 2026.
[11] A. Memon, Z. Gao, B. Nguyen, S. Dhanda, E. Nickell, R. Siemborski, and J. Micco, "Taming Google-scale continuous testing," in *Proc. IEEE/ACM Int. Conf. Software Engineering in Practice (ICSE-SEIP)*, 2017.
[12] A. Mesbah, A. van Deursen, and S. Lenselink, "Crawling Ajax-based web applications through dynamic analysis of user interface state changes," *ACM Trans. Web*, vol. 6, no. 1, 2012.
[13] S. R. Choudhary, H. Versee, and A. Orso, "WEBDIFF: Automated identification of cross-browser issues in web applications," in *Proc. IEEE Int. Conf. Software Maintenance (ICSM)*, 2011.
[14] F. Ricca, M. Leotta, and A. Stocco, "Three open problems in the context of e2e web testing and a vision: NEONATE," *Advances in Computers*, vol. 113, 2019.
[15] S. Panichella, C. Aaron Imtiaz, and H. C. Gall, "Test smells in software test automation: A systematic mapping study," *IEEE Access*, 2019.
[16] A. Bertolino, G. De Angelis, M. Kwiatkowski, B. Miranda, and S. Sabetta, "The quest for software test effectiveness measures," *IEEE Software*, 2021.
[17] L. Briand, D. Bianculli, S. Nejati, F. Pastore, and M. Sabetzadeh, "The case for context-driven software engineering research," *IEEE Software*, vol. 34, 2017.
[18] A. Arcuri and L. Briand, "A Hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering," *Softw. Test., Verif. Reliab.*, vol. 24, no. 3, 2014.
[19] M. Grechanik, Q. Xie, and C. Fu, "Maintaining and evolving GUI-directed test scripts," in *Proc. IEEE Int. Conf. Software Engineering (ICSE)*, 2009.
[20] Q. Xie and A. Memon, "Using a pilot study to derive a GUI model for automated testing," *ACM Trans. Softw. Eng. Methodol.*, vol. 18, no. 2, 2008.